
Planning & Control

2026, July

SuHan Yoon

yshgit@gmail.com

목차

■ 제어 이론 (Control Theory)

- ▶ 제어 개념 · Open / Closed-Loop · 전달함수 · 극점과 안정성 · s · z · w 도메인

■ 차량 모델 (Vehicle Model)

- ▶ 조향 방식 · Kinematic Bicycle Model · RPY

■ 피드백 제어 (Feedback Control)

- ▶ 종방향 제어 · PID Control

■ 경로 추종 (Path Tracking)

- ▶ Pure Pursuit · Stanley · 기법 비교

■ 경로 생성 & 판단 (Path & Planning)

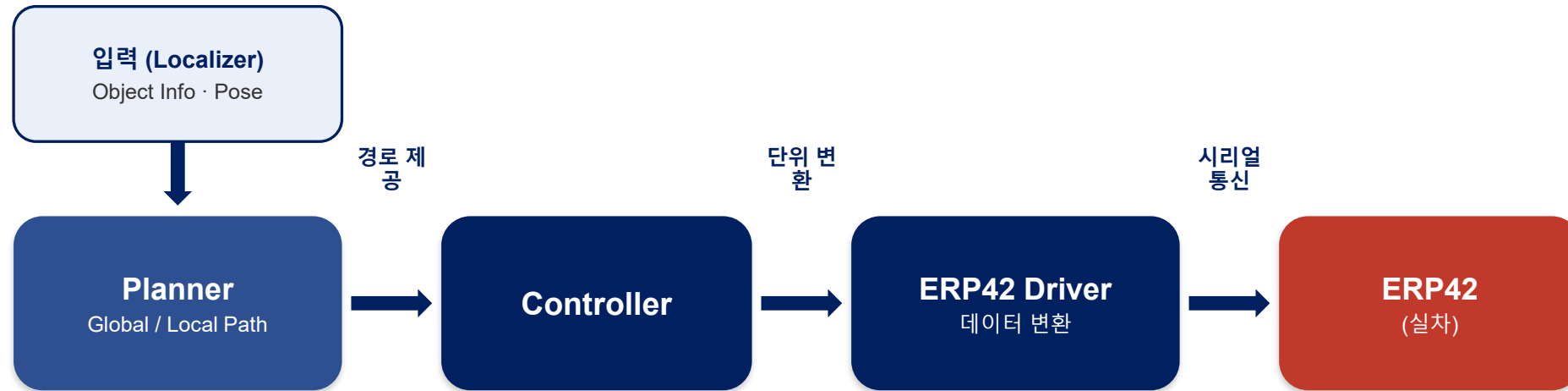
- ▶ Global Path 생성 · 계획 계층 · 행동계획 · 신호등 탐지

■ 실습 & Tip

- ▶ ERP-Driver · 토픽 타입 변경 · rqt · 실습 미션

Architecture

- Planner가 경로를 생성하고 Controller가 ERP42에 맞게 변환하여 차량을 구동



PART 1 · 제어 이론

Control

■ Morai → ERP 토픽 변경

- ▶ pure_pursuit_pid_velocity_planning.py
- ▶ erp_driver – ErpSerialHandler.py

■ /erp42_status 에서 speed 를 받아온다

- ▶ 받은 status를 제어에 사용한다

```
self.alive = 0
self.packet = erpCmdMsg()
self.packet.gear = 0
self.packet.e_stop = False
self.packet.brake = 1

self.erpMotionMsg_pub = rospy.Publisher(
    "/erp42_status", erpStatusMsg, queue_size=3
)
self.erpCmdMsg_sub = rospy.Subscriber(
    "/erp42_ctrl_cmd", erpCmdMsg, self.sendPacket
)

def recvPacket(self) -> None:
    packet = self.serial.read(18)
```

```
self.current_position.msg.pose.position.y
self.current_position.y=msg.pose.pose.position.y

def status_callback(self,msg): ## Vehicle Status Subscriber
    self.is_status=True
    self.status_msg=msg
```

제어(Control)란 무엇인가?

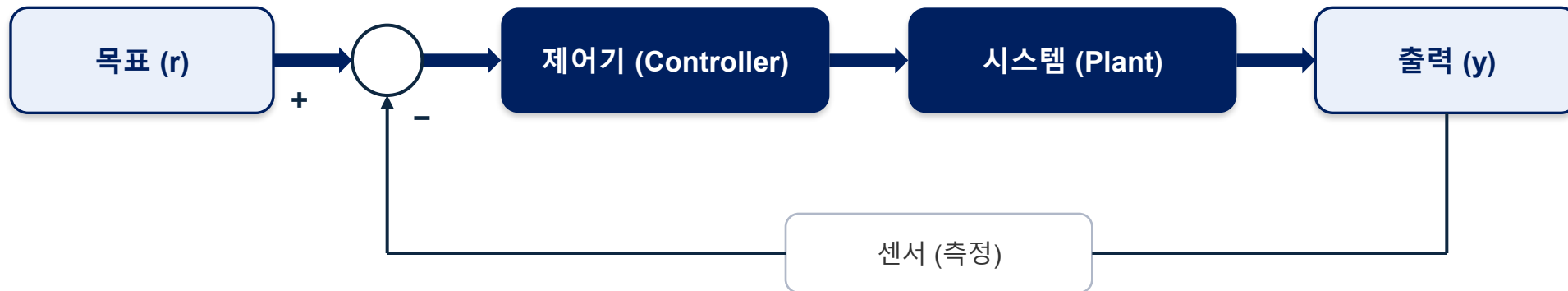
■ 정의

- ▶ 시스템의 출력(y)이 원하는 목표(r)와 일치하도록 입력(u)을 계산·조절하는 과정
- ▶ 핵심 원리 : 측정 $\rightarrow$ 비교(오차 $e = r - y$) $\rightarrow$ 보정의 반복

■ 목표

- ▶ 더 빠르게(속응성) · 더 정확하게(정상상태 오차 $\downarrow$) · 더 안정적으로(진동 없이) 목표를 추종

피드백 제어 구조 (Closed-Loop Control)



일상 속 제어 예시

에어컨 : 실내 온도 유지

크루즈 컨트롤 : 차량 속도 유지

드론 자세 제어 : 수평 자세 복원

제어 방식의 두 갈래 — Open-Loop vs Closed-Loop

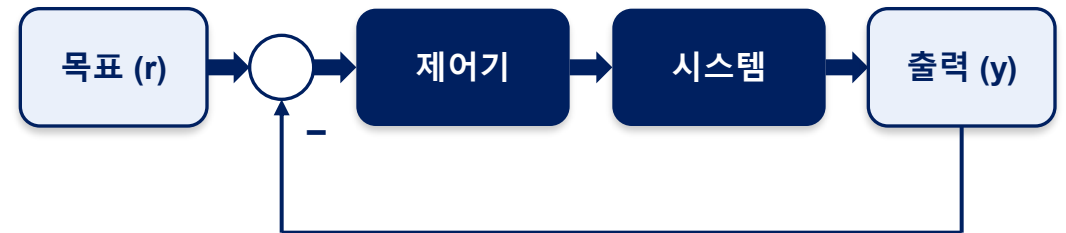
- Open-Loop : 출력을 보지 않고, 미리 정한 입력만 내보내는 방식
- Closed-Loop : 출력을 측정해 목표와의 오차를 계속 보정하는 방식

Open-Loop (개루프)



장점 : 구조 단순 · 저렴 · 설계 쉬움
한계 : 외란이 생겨도 알아채지 못함

Closed-Loop (폐루프)



장점 : 오차 자동 보정 · 외란에 강함
대가 : 센서 필요 · 설계 복잡

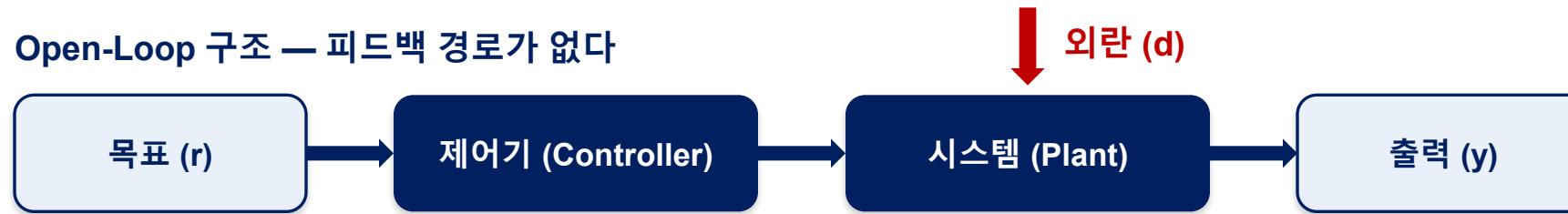
차이는 단 하나 — 출력을 다시 보는가 (Feedback 유무)

Open-Loop 제어 (개루프)

■ 정의

- ▶ 출력을 측정하지 않고, 미리 정해진 입력을 그대로 인가하는 제어
- ▶ 외란·모델 오차가 생겨도 시스템 스스로 알아채지 못함

Open-Loop 구조 — 피드백 경로가 없다



장점

구조 단순 · 비용 저렴 · 센서 불필요
설계가 쉽고 응답이 빠름

한계

외란·부하 변화를 보정할 수 없음
모델이 부정확하면 오차가 그대로 누적

일상 예시

세탁기 : 정해진 시간만 동작

토스터 : 시간이 지나면 종료

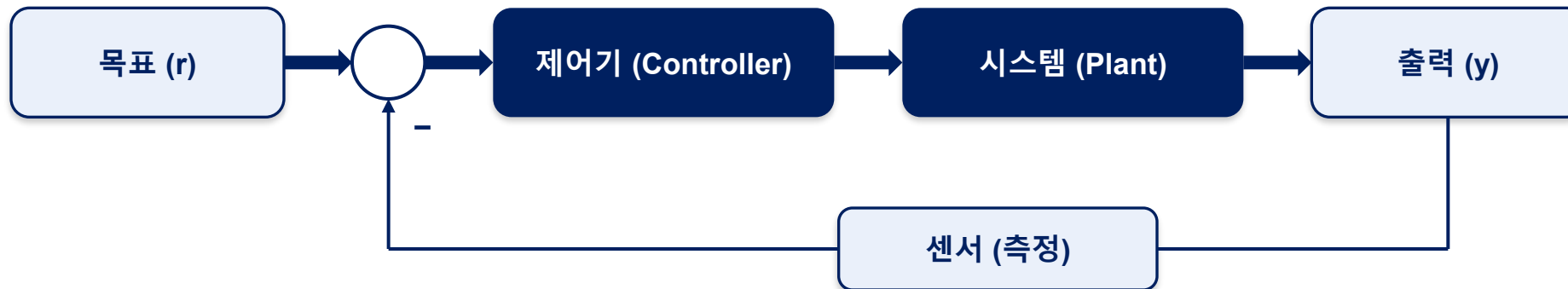
선풍기 : 단수만큼만 회전

Closed-Loop 제어 (폐루프)

■ 정의

- ▶ 출력을 센서로 측정하고 목표와 비교해, 오차($e = r - y$)로 입력을 보정
- ▶ 외란이 와도 오차로 드러나기 때문에 자동으로 대응 가능

Closed-Loop 구조 — 오차가 계속 순환하며 보정된다



장점

외란 억제 · 목표 자동 추종 · 모델 오차에 강함

대가

센서·연산 비용 · 잘못 설계하면 진동·발산

일상 예시

크루즈 컨트롤 : 속도 유지

에어컨 : 온도 유지

드론 : 수평 자세 유지

Open-Loop vs Closed-Loop 비교

- 같은 목표를 다뤄도 두 방식의 성격은 뚜렷하게 다르다

항목	Open-Loop (개루프)	Closed-Loop (폐루프)
구조	단순 (피드백 없음)	복잡 (센서 + 비교기 + 제어기)
정확도	모델 정확도에 전적으로 의존	오차를 보정하므로 높음
외란 대응	불가	자동 보정
비용	저렴	센서·연산 비용 발생
안정성 위험	발산 위험 없음	잘못 설계하면 진동·발산 가능
대표 예시	세탁기 타이머 · 토스터	크루즈 컨트롤 · 드론 호버링

자율주행의 종·횡방향 제어는 모두 Closed-Loop 기반

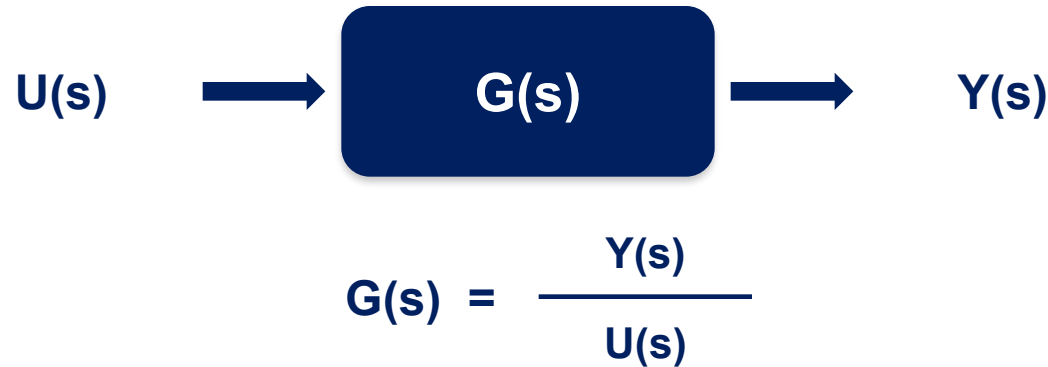
전달함수 (Transfer Function) 기초

■ 라플라스 변환 (Laplace Transform)

- ▶ 시간영역의 미분방정식을 s영역의 대수식(곱셈·나눗셈)으로 바꿔줌

■ 전달함수 $G(s)$

- ▶ 입력 $U(s)$ 대비 출력 $Y(s)$ 의 비 — 시스템의 입출력 관계를 분수식 하나로 표현

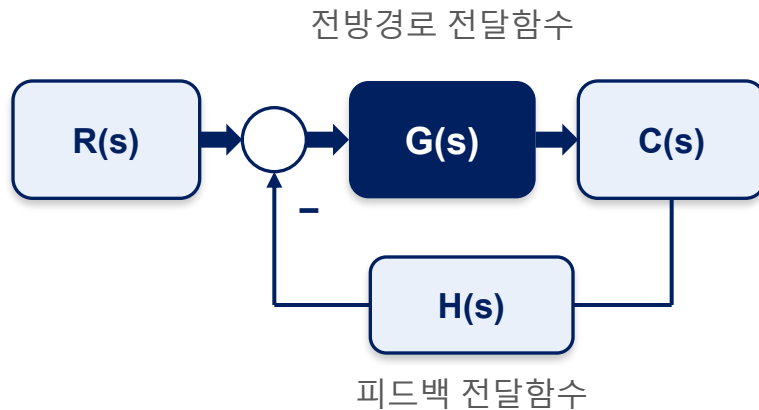


예) RC 회로 : $G(s) = 1 / (RCs + 1)$ 질량-스프링-댐퍼 : $G(s) = 1 / (ms^2 + cs + k)$

미분방정식이 곱셈·나눗셈이 된다 — 그래서 설계는 s영역에서 한다 (시간영역에서는 컨볼루션)

Closed-Loop 전달함수 유도

- 블록선도를 식으로 옮기면 폐루프 전달함수가 유도된다



① 오차 : $E(s) = R(s) - H(s) \cdot C(s)$

② 출력 : $C(s) = G(s) \cdot E(s)$

③ 정리 : $C = G(R - HC) \rightarrow (1 + GH) \cdot C = G \cdot R$

$$T(s) = C(s) / R(s) = G(s) / (1 + G(s)H(s))$$

개루프 전달함수 $L(s) = G(s)H(s)$ — 피드백을 끊고 루프를 한 바퀴 돈 이득 (안정성 판별의 기준)

특성방정식 — 식이 말해주는 것

■ $T(s) = G / (1 + GH)$ — 이 식의 성질은 전부 분모가 결정한다

① 특성방정식

$$1 + GH = 0$$

이 방정식의 근(= 극점)이 수렴·발산 등 시스템의 운명을 결정

② 감도 (외란 억제)

$$S = 1 / (1 + GH)$$

루프 이득 GH가 클수록 외란·모델 변화의 영향이 줄어든다

③ 정상상태 오차

$$E = R / (1 + GH)$$

루프 이득을 키우면 목표와의 남는 오차가 작아진다

폐루프의 모든 성질은 분모 $1 + GH$ 가 결정한다

다음 → 그 근(극점)의 위치에 따라 응답이 수렴하거나, 진동하거나, 발산한다

극점 p는 어디서 왔는가 — 전달함수의 분모

■ 다음 페이지의 출발점 $dy/dt = p \cdot y$ — 그 p의 고향은 전달함수의 분모다

① 극점의 고향 — 분모 $G(s) = Y(s)/U(s) = 1 / (s - p)$
분모를 0으로 만드는 값 $s = p$ 가 극점 — 그 p가 바로 여기서 온 상수

② s 영역에서 전개 $(s - p) \cdot Y(s) = U(s) \rightarrow s \cdot Y(s) - p \cdot Y(s) = U(s)$
분수를 없애고 양변을 풀어 쓴다

③ s를 미분으로 되돌리기 $s \cdot Y(s) \rightarrow dy/dt$, $p \cdot Y(s) \rightarrow p \cdot y(t)$, $U(s) \rightarrow u(t)$
'미분 = s 곱하기' 규칙을 거꾸로 적용 $\rightarrow dy/dt - p \cdot y(t) = u(t)$

④ 입력이 없다면? (자유 응답) $u(t) = 0 \rightarrow dy/dt = p \cdot y(t)$
외부 힘을 끄면 시스템 고유의 움직임만 남는다 — 극점이 만드는 순수한 응답

분모의 극점 p가 시간영역의 비례상수로 복원된다 — 그래서 $y(t) = K \cdot e^{pt}$ 의 지수에 p가 박힌다

극점은 왜 지수함수를 낳는가 — $y(t) = Ke^{pt}$

■ 가장 단순한 1차 시스템을 직접 풀어보면 — 극점 p 가 지수함수의 지수로 들어간다

① 미분방정식 세우기 $dy/dt = p \cdot y$

앞 페이지에서 유도한 자유 응답 식 — 변하는 속도가 현재 상태에 비례하고, 그 비례상수가 극점 p

② 변수 분리 · 적분 $\int (1/y) dy = \int p dt \rightarrow \ln|y| = pt + C$

y 는 좌변, t 는 우변으로 — $1/y$ 의 적분은 자연로그 $\ln$

③ 지수함수로 변환 $y(t) = e^{(pt+C)} = K \cdot e^{(pt)}$

양변을 e 의 지수로 올려 $\ln$ 을 벗김 — e^C 는 상수이므로 초기값 K 로 치환

결과 읽기

$$y(t) = K \cdot e^{(pt)}$$

$p < 0 \rightarrow 0$ 으로 감쇠 : 수렴

$p = 0 \rightarrow K$ 그대로 : 임계

$p > 0 \rightarrow$ 무한대로 : 발산

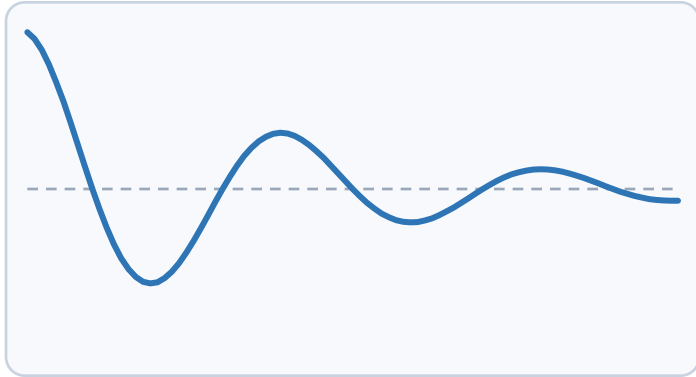
$K =$ 초기값 $y(0)$

극점 p 는 $e^{(pt)}$ 의 지수 — 그래서 극점의 위치가 응답의 운명을 결정한다

시스템 응답 – 수렴 · 임계 · 발산

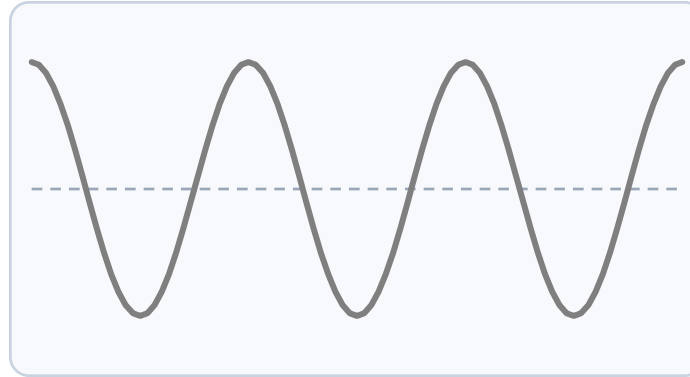
■ 특성방정식의 근(극점)의 실수부 부호에 따라 시스템 응답의 운명이 갈린다

수렴 (Stable)



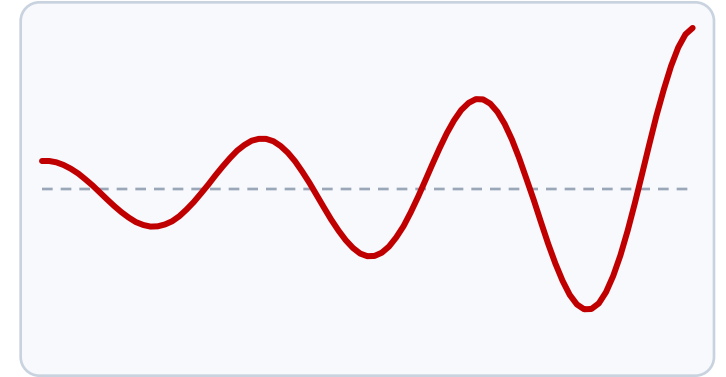
진동이 잦아들며 목표에 정착
 $\text{Re}(s) < 0$ (안정)

임계 (Marginally Stable)



같은 진폭으로 진동이 지속
 $\text{Re}(s) = 0$ (임계 안정)

발산 (Unstable)



진폭이 점점 커져 제어 실패
 $\text{Re}(s) > 0$ (불안정)

극점의 위치가 시스템의 운명을 가른다

제어기 설계란 곧, 극점을 안정 영역으로 옮기는 일이다

왜 도메인이 나누는가 — 연속과 이산

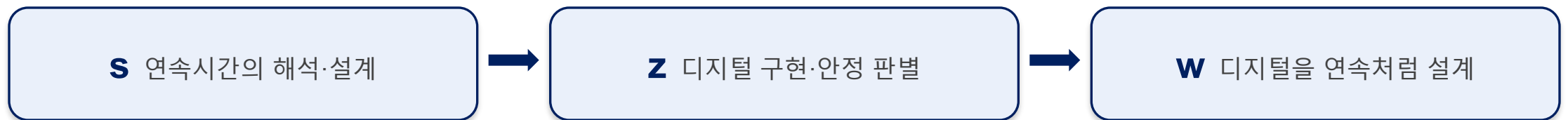
■ 물리 세계와 컴퓨터의 세계

- ▶ 차량·모터 등 물리 시스템은 연속시간(아날로그) — 라플라스 변환, s -도메인
- ▶ 제어기는 컴퓨터 — 샘플링 주기 T 마다 계산하는 이산시간(디지털), z -도메인

디지털 제어 루프 — 연속과 이산이 한 루프 안에 공존한다



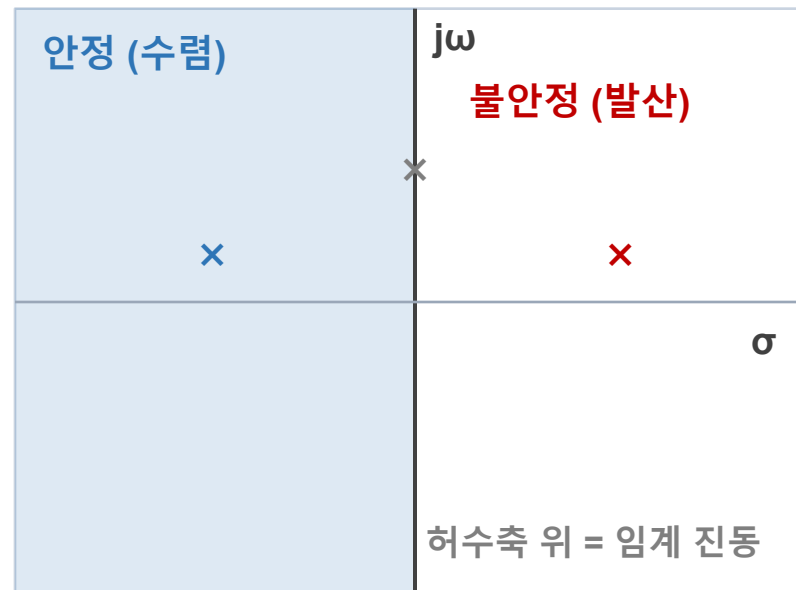
그래서 도메인이 세 개 필요하다



s-도메인 — 연속시간의 안정성

- ▶ 연속시간 신호·시스템을 라플라스 변환으로 해석 — 복소변수 $s = \sigma + j\omega$
- ▶ 안정 조건 : 모든 극점의 실수부가 음수 — 극점이 좌반평면(LHP) 안에 있어야 한다

s-평면 (복소평면)



읽는 법

- x 표시가 극점의 위치
- 왼쪽 깊숙할수록 빨리 수렴
- 축에서 위아래로 멀수록 빠른 진동

예시

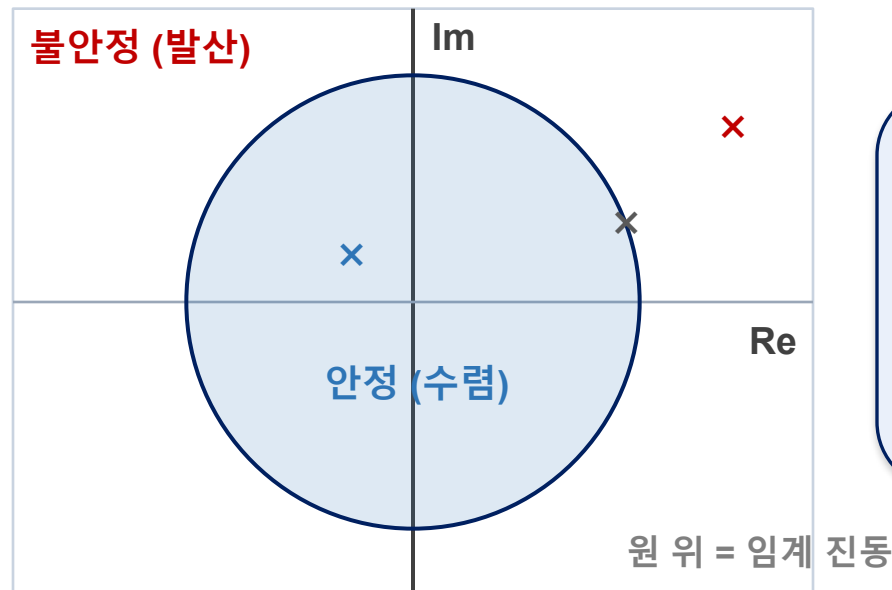
$G(s) = 1/(s+2) \rightarrow$ 극점 -2 : 수렴
 $G(s) = 1/(s-1) \rightarrow$ 극점 $+1$: 발산

s-평면의 왼쪽 절반이 '안전지대' — 설계는 극점을 이 안에 넣는 일

z-도메인 — 이산시간의 안정성

- ▶ 샘플링(주기 T)된 이산시간 신호를 z-변환으로 해석 — 두 평면의 매핑은 $z = e^{sT}$
- ▶ 안정 조건 : 모든 극점이 단위원 내부 ($|z| < 1$) 에 있어야 한다

z-평면 (복소평면)



읽는 법

- 단위원은 s-평면 허수축의 짝
- 원 안 = s의 좌반평면(안정)
- 원점에 가까울수록 빨리 수렴

매핑 $z = e^{sT}$

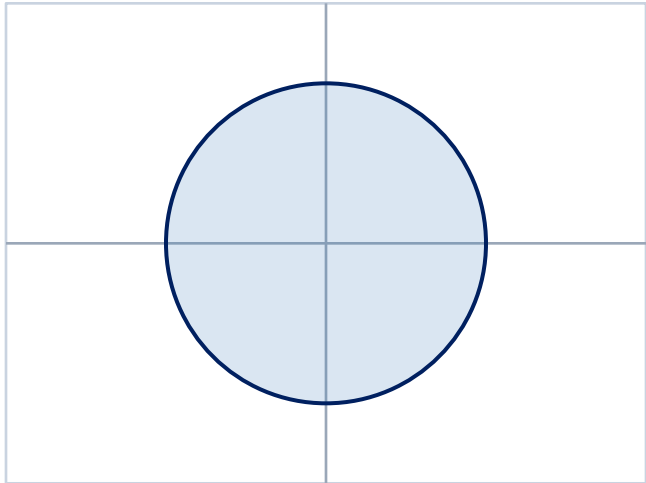
- $s = 0 \rightarrow z = 1$ (경계끼리 대응)
- $\text{Re}(s) < 0 \rightarrow |z| < 1$: 원 안 (안정)
- $\text{Re}(s) > 0 \rightarrow |z| > 1$: 원 밖 (발산)

디지털에서는 원이 기준 — 극점이 단위원 안에 있어야 안정

w-도메인 — 디지털을 연속처럼 설계

- ▶ 쌍선형(Tustin) 변환으로 z-평면을 다시 s-평면과 같은 모양으로 펴준 도메인
- ▶ 단위원 내부 ↔ w-평면 좌반평면 — 보드선도 등 연속 설계기법을 디지털에 그대로 사용

z-평면 : 단위원 내부가 안정



$$w = (2/T) \cdot (z-1)/(z+1)$$



쌍선형 (Tustin) 변환

w-평면 : 좌반평면이 안정



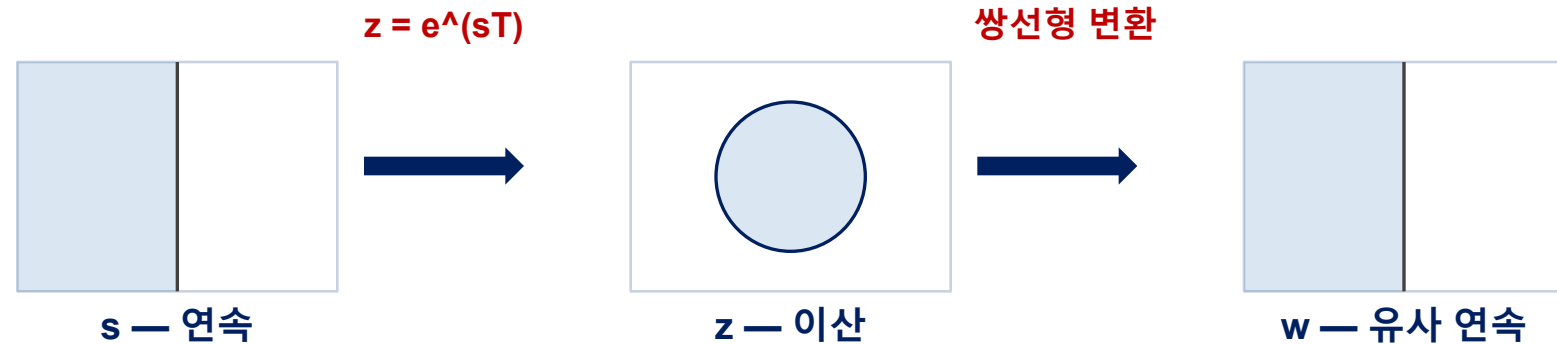
주의 : 주파수 워핑(warping) — 높은 주파수 눈금이 뒤틀린다 → 중요 주파수는 프리워핑으로 보정

w 덕분에 — 연속에서 배운 설계법을 디지털에서도 그대로 쓴다

도메인 요약 — $s \cdot z \cdot w$

■ 세 도메인은 하나의 시스템을 보는 세 개의 창 — 역할이 다를 뿐이다

도메인	변환	안정 영역	주 용도
s	라플라스 변환	좌반평면 $\text{Re}(s) < 0$	연속 시스템 해석·설계
z	z-변환 ($z = e^{sT}$)	단위원 내부 $ z < 1$	디지털 구현·안정 판별
w	쌍선형 (Tustin) 변환	좌반평면 $\text{Re}(w) < 0$	디지털을 연속기법으로 설계



설계는 $s \cdot w$ 에서, 구현은 z 에서 — 이제 실제 차량 제어(PID)로 간다

PART 2 · 차량 모델 & 제어

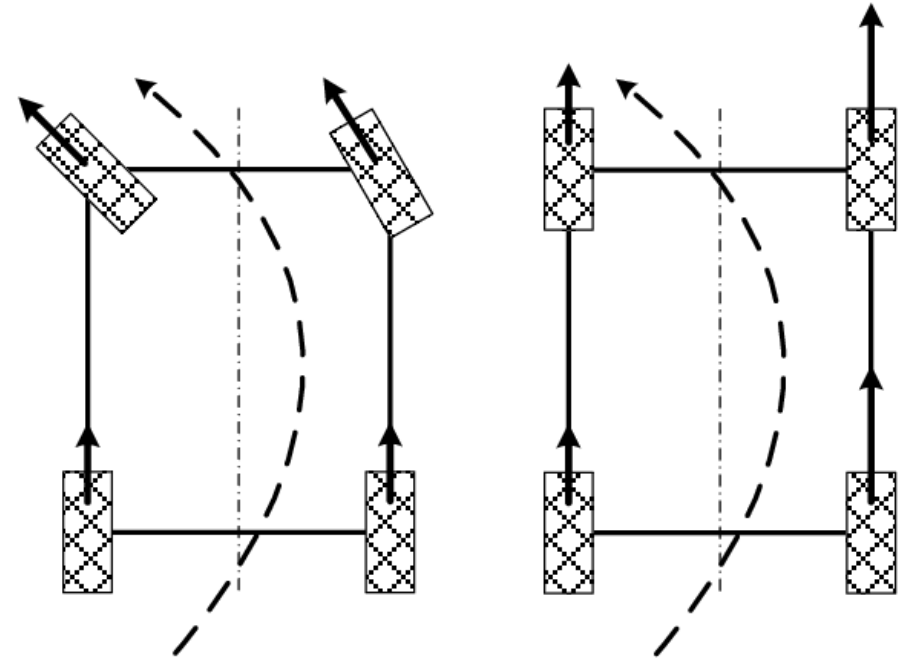
Vehicle Model – 조향 방법

■ Ackermann – ERP 방식

- ▶ 각 바퀴의 각도를 바꿔서 조향

■ Differential

- ▶ 각 바퀴의 속도를 바꿔서 조향



(a) Ackerman-steered vehicle

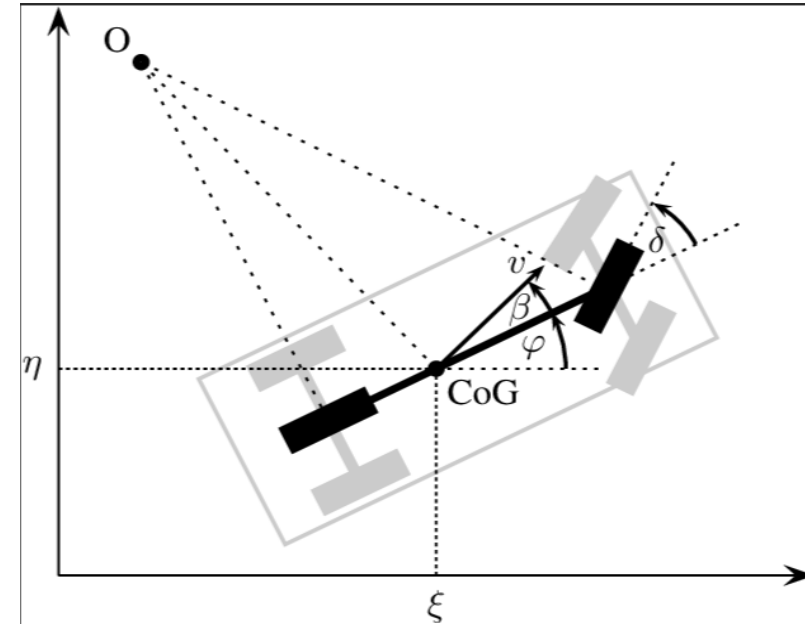
(b) Differential-steered vehicle

(a) Ackermann-steered (b) Differential-steered

Vehicle Model – Kinematic Bicycle Model

■ Kinematic Bicycle Model 이란?

- ▶ 차량의 위치·자세·속도를 단순한 수학적 모델로 표현하는 대표 모델
- ▶ Ackermann Model을 단순화한 버전
- ▶ 현재 ERP42가 사용하는 모델



$$\dot{x} = v \cos \theta$$

$$\dot{y} = v \sin \theta$$

$$\dot{\theta} = \frac{v}{L} \tan \delta$$

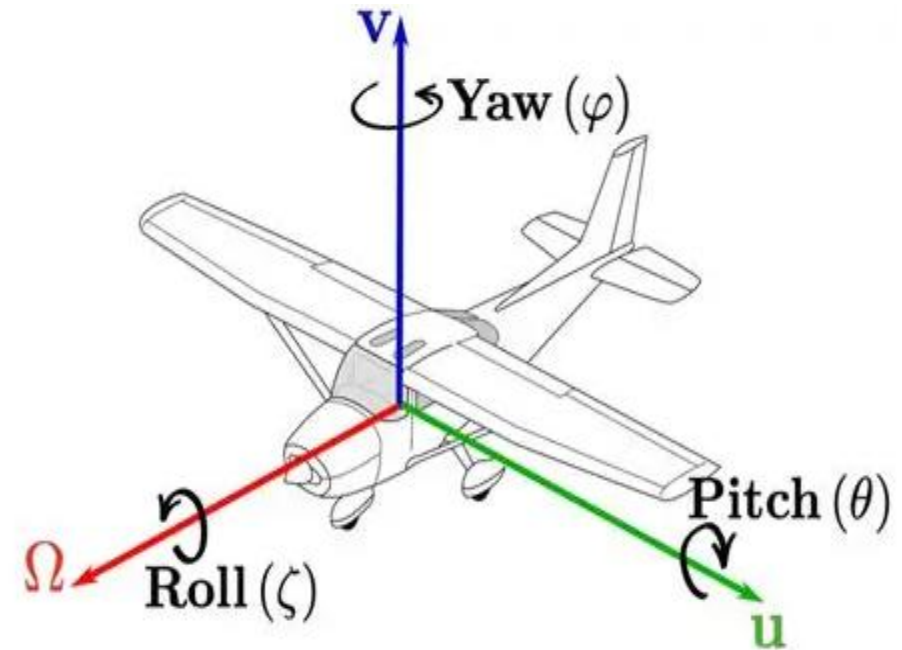
Vehicle Model – RPY (Roll · Pitch · Yaw)

■ RPY (Roll · Pitch · Yaw)

- ▶ Roll : x축을 기준으로 회전
- ▶ Pitch : y축을 기준으로 회전
- ▶ Yaw : z축을 기준으로 회전

■ 차량에서는 보통 Yaw가 압도적으로 많이 사용

- ▶ Localizer에 Yaw를 계산하는 수식 존재



Feedback Control

■ Feedback Control 이란?

- ▶ 시스템 출력을 측정해 목표와 비교하고, 차이를 줄이도록 입력을 조절하는 방식

■ 피드백 제어가 없는 경우

- ▶ 자동으로 오차를 줄이지 못한다
- ▶ 외란(disturbance)에 취약해진다
- ▶ 정확도가 낮아진다

⇒ 따라서 Feedback Control은 사실상 필수

Longitudinal Feedback Control – PID Control 이란?

- 비례·적분·미분 세 가지 요소로 시스템 오차를 보정하여 목표값을 얻는 제어 방법

$$u(t) = K_p \left(e(t) + \frac{1}{T_i} \int_0^t e(\tau) d\tau + T_d \frac{d}{dt} e(t) \right)$$

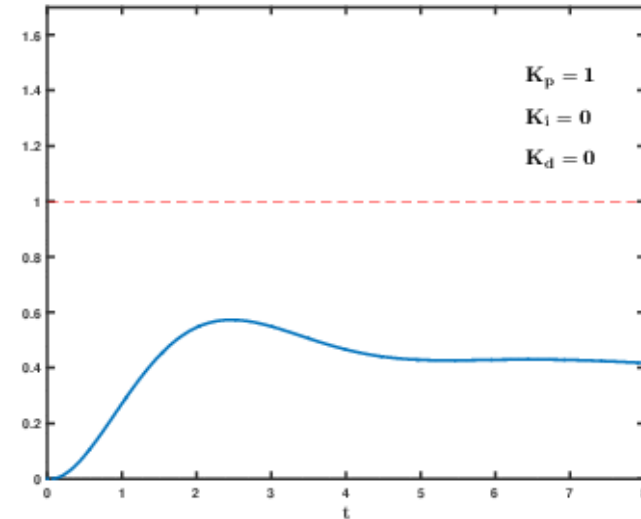
항	이름	역할	단점
P	비례 (Proportional)	오차가 클수록 크게 반응 → 빠른 반응	오차를 완전히 없애지는 못함
I	적분 (Integral)	누적된 오차를 보정 → 정확한 수렴	오차 누적 시 과도 반응 (오버슈트)
D	미분 (Derivative)	오차 변화를 미리 감지 → 진동 억제	노이즈에 민감하여 출력이 흔들림

Longitudinal Feedback Control – PID Control – 조정 방법

■ pure_pursuit_pid_velocity_planning.py

■ 튜닝 순서

- ▶ p_gain 만 활성화 후 기본 반응 확인
- ▶ p_gain 을 조절해 원하는 반응 속도까지 조정
- ▶ 진동이 발생하면 D 를 추가
- ▶ 줄어든 정확도는 I 를 추가하여 오차 제거
- ▶ 필요 시 dt 조정 ($dt \downarrow \rightarrow$ 반응 빠름, $I \downarrow$, $D \uparrow$)



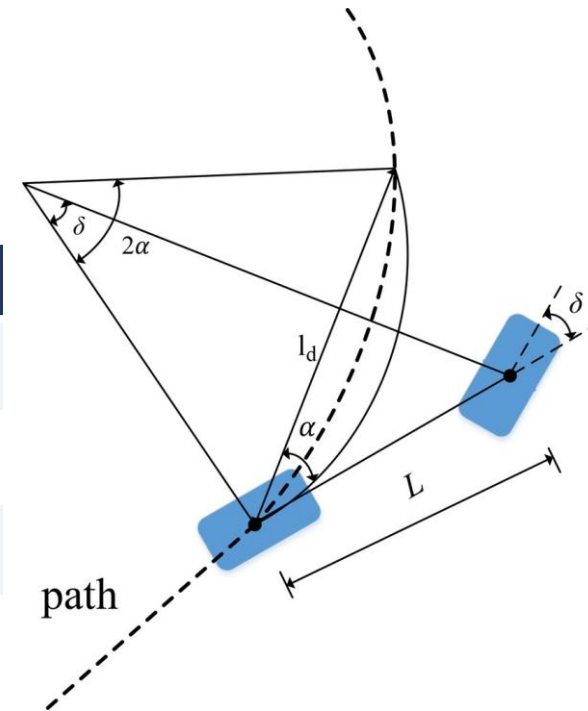
```
class pidControl:
    def __init__(self):
        self.p_gain = 0.3
        self.i_gain = 0.00
        self.d_gain = 0.03
        self.prev_error = 0
        self.i_control = 0
        self.controlTime = 0.02
```

Lateral Feedback Control – Pure Pursuit

■ Pure Pursuit 이란?

- ▶ 현재 위치에서 일정 거리 앞의 목표점을 바라보고 그 점을 추종하도록 유도하는 방식

구성 요소	기능 설명
Ld	Look-ahead 거리 (LFD)
L	차량의 휠베이스 (앞바퀴 – 뒷바퀴 거리)
α	차량과 LFD 점을 향한 벡터 사이의 각도



Pure Pursuit

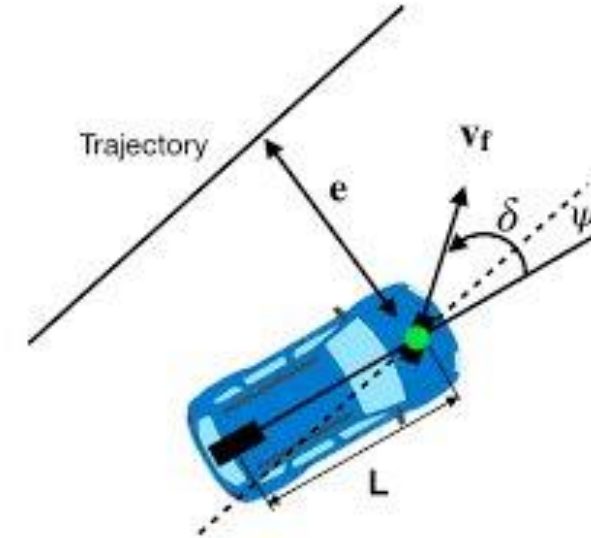
$$\delta = \arctan \frac{e_y}{L}$$

Lateral Feedback Control – Stanley

■ Stanley 이란?

- ▶ 경로와 수직인 거리를 횡방향 오차로 두어, 차량이 경로 중심을 따라가도록 조향각을 제어하는 기법

구성 요소	기능 설명
θ_e	차량 방향과 경로 방향의 차이 보정
e_y	경로 중심에서 벗어난 거리 보정
k	경로로 빠르게 복귀하는 민감도 조절
v	속도가 높을수록 조향각을 줄이는 효과



Stanley

$$\delta = \arctan \frac{ke_y}{v + \varepsilon}$$

경로 추종 기법 비교 (Pure Pursuit vs Stanley)

Stanley 조향각

$$\delta = \theta_e \tan^{-1} \left(\frac{k \cdot e_c}{v} \right)$$

Pure Pursuit 조향각

$$\delta = \tan^{-1} \left(\frac{2L \cdot \sin \alpha}{L_d} \right)$$

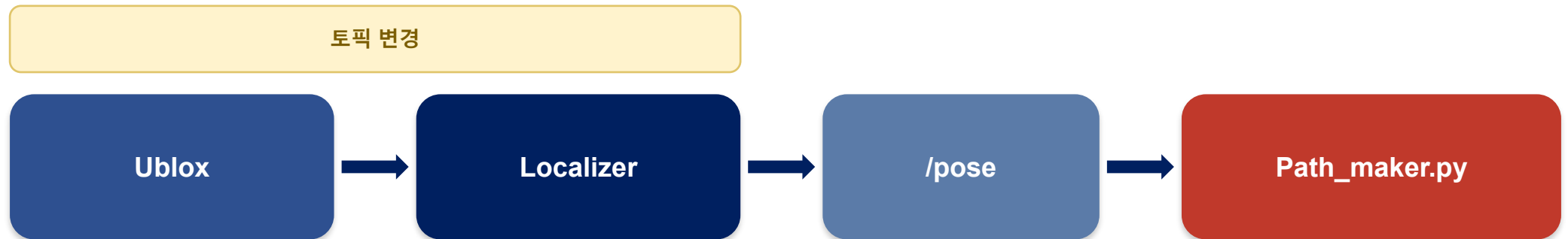
비교 항목	Stanley 제어기	Pure Pursuit 제어기
기본 원리	수직 방향 오차 + 헤딩 오차 이용	일정 거리 앞 Look-ahead Point 추적
장점	고속 주행 시 안정적 경로에 수직 수렴 → 정밀 추종 헤딩 오차 반영 → 회전 안정	구현이 매우 단순 연산량이 작아 실시간 적합 저속에서도 부드러운 추종
단점	저속 시 진동 발생 가능 파라미터 k 튜닝 필요	고속 시 look-ahead 거리에 민감 급커브 오버슈트·언더슈트 정밀 추종 어려움
속도 영향	내재적 속도 반영 (고속→조향각↓)	속도에 따라 LFD 조정 필요

PART 3 · 경로 생성 & 판단

Path 생성

■ 토픽 변경

- ▶ 모라이 기준으로 되어 있는 토픽들을 ERP 기준으로 변경
- ▶ 그 이후 모라이에서와 같이 path 를 생성



HOW TO CREATE A GLOBAL PATH · 개요

■ Offset

- ▶ 지도상의 기준위치와 실제 위치 간의 차이를 의미 — 데이터마다 기준 좌표계가 다르다



HD Map 생성

HD Map
Map을 생성하기 시작한 위치가 기준



GPS 데이터

GPS Data
GPS 좌표계가 기준



기타 데이터

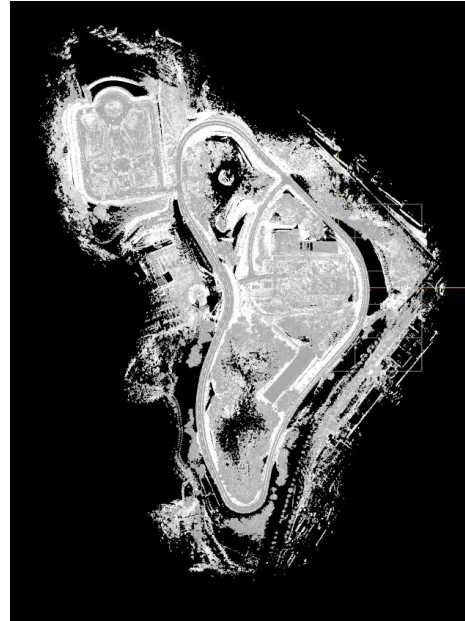
ETC Data
각각 자기 기준을 적용

⇒ 서로 다른 좌표계 기준을 가지고 있다면 **HD Map 생성이 어려움**

HOW TO CREATE A GLOBAL PATH · ① 좌표계 문제

≠

PCD 기준
LOCAL COORDINATE
X = 200
Y = 300



GPS 기준
GLOBAL (UTM) COORDINATE
X = 361025.8208815998
Y = 4066196.298577933

⇒ 두 좌표계는 원점이 달라 하나의 map으로 합칠 수 없다 — **Offset이 필요하다**

HOW TO CREATE A GLOBAL PATH · ② Offset 취득

- **MORAI** 시뮬레이터 우측 하단을 클릭하면 map의 **offset** 값이 클립보드에 복사된다



```
{"isUtm":true,"utmNum":52,"utmOffset":{"x":360825.8208815998,"y":4065896.298577933,"z":0.0},"projData":{"SPHEROID":"WGS84","LatitudeOfOrigin":0.0,"CentralMeridian":0.0,"ScaleFactor":1.0,"FalseEasting":0.0,"FalseNorthing":0.0}}
```

복사한 offset을 붙여넣으면 **PCD ↔ GPS 좌표 변환 기준**이 완성된다

HOW TO CREATE A GLOBAL PATH · ③ 적용 결과

- MORAI Offset 적용 — 두 좌표계가 하나의 map으로 정합된다

모라이 기반 Path

MORAI simulator



호환 가능

실제 GPS 기반 Path

Real-world GPS

MORAI로 Global Path 취득 시 이점



GPS 오차 영향 감소

실측 대신 정합된 map 기준으로 path를 생성 → 위치 오차로 인한 경로 오류 감소



야외 실험 최소화

실차(ERP42) 없이 시뮬레이터만으로 Global Path 확보 → 비효율적 현장 실험 감소

시뮬레이터만으로 실차 수준의 Global Path 확보 → 개발 효율 향상

경로 계획 계층 (Planning Hierarchy)

■ 경로 계획은 상위 → 하위로 갈수록 경로를 점점 구체화 (coarse → fine)



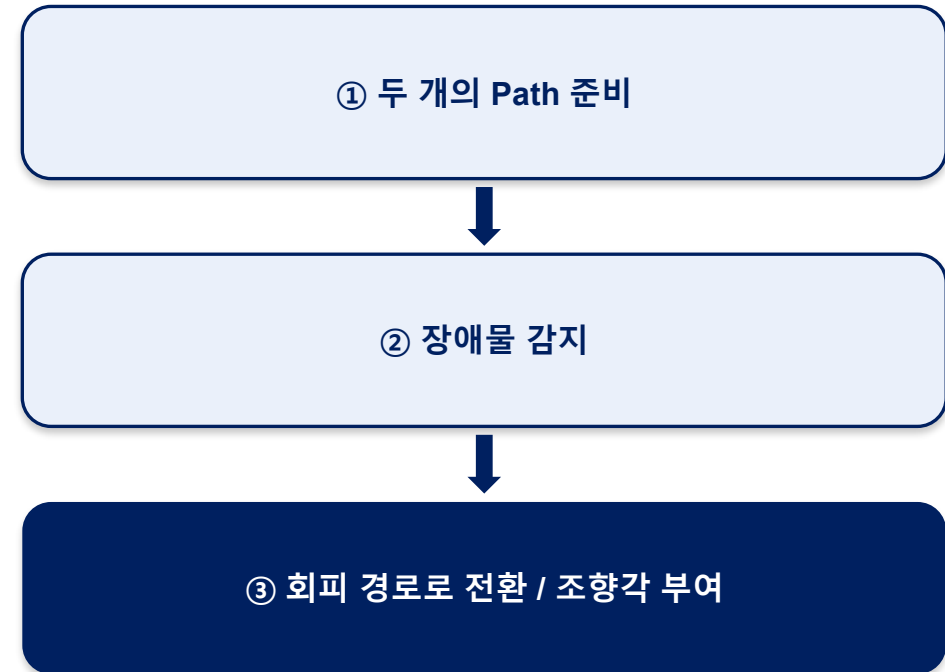
판단 – 행동계획

■ 예시 – LaneChange

- ▶ Path 를 두 개 생성해 장애물이 있는 경우 Path 를 변경

■ 예시 – 지역경로

- ▶ Lattice Planning
- ▶ A*
 - 휴리스틱을 장애물과의 거리



판단 – 신호등 탐지

■ 예시 – 신호등 탐지

- ▶ 빨간불을 탐지했을 경우 True 를 publish 하도록 인지부를 조정 후 subscribe
- ▶ 제어부를 다음과 같이 수정

```
# 빨간불 탐지 시 True를 pub하도록 인지부 조정 후 sub
def yolo_callback(self, msg):
    self.red_flag = msg    # 클래스 멤버 변수로 저장

# 속도를 정하는 곳에 if문 생성
if self.red_flag:
    cmd.speed = 0
```

PART 4 · 실습 & 마무리

To Do

■ 제어 해야 할 것

- ▶ 모라이 전용 토픽 → ERP 토픽으로 변경
- ▶ ERP 에 맞도록 속도·조향 단위 변경
- ▶ GNSS값 pose로 변경
- ▶ 주행 테스트를 진행하며 PID 등 파라미터 조정

■ 판단 해야 할 것

- ▶ 모라이 혹은 실차에서 Path 생성 (Localizer에서 나오는 토픽 pose 이용)
- ▶ 장애물 탐지 및 회피 알고리즘 생성

ERP-Driver

■ Controller 출력 → ERP 단위 변환

- ▶ $\text{steer} = \text{degree} \rightarrow (-1) \times \text{degree} \times 71$ 후 publish
 - 왼쪽이면 -, 오른쪽이면 +
- ▶ $\text{speed} = \text{kph} \rightarrow \text{kph} \times 10$ 후 publish

UPPER to PCU														
	Packet (14Byte)													
BYTE NAME	S	T	X	AorM	E-STOP	GEAR	SPEED0	SPEED1	STEER0	STEER1	BRAKE	ALIVE	ETX0	ETX1
값	0x53	0x54	0x58	0x00 or 0x01	0x00 or 0x01	0~2	0~200		-2000~2000		1~33	0~255	0x0D	0x0A

PCU to UPPER																		
Packet (18Byte)																		
BYTE NAME	S	T	X	AorM	E-STOP	GEAR	SPEED0	SPEED1	STEER0	STEER1	BRAKE	ENC0	ENC1	ENC2	ENC3	ALIVE	ETX0	ETX1
값	0x53	0x54	0x58	0x00 or 0x01	0x00 or 0x01	0~2	0~200		-2000~2000		1~33	-2^31~-2^31				0~255	0x0D	0x0A

AorM -> Auto or Manual, 0x00 : manual mode , 0x01 : auto mode
 ESTOP -> Emergency STOP, 0x00 : E-STOP Off, 0x01 : E-STOP On
 GEAR -> 0x00 : forward drive, 0x01 : neutral, 0x02 : backward drive
 SPEED -> actual speed (KPH) * 10
 STEER -> actual steering degree (degree) * 71, 오차율 : 4%, negative is left steer
 BRAKE -> 1 : no braking, 33 : full braking
 ENC -> encoder counting
 ALIVE -> increasing each one step

ERP-42 Packet 구조

```

theta = atan2(local_path_point[1],local_path_point[0])
steering = atan2((2*self.vehicle_length*sin(theta)),self.lfd)

return steering
    
```

```

if output > 0.0:
    self.ctrl_cmd_msg.accel = output
    self.ctrl_cmd_msg.brake = 0.0
    
```

Tip – 토픽 타입 바꾸는 방법 (geometry_msgs)

■ 1. Import

- ▶ `from geometry_msgs.msg import xxx`
- ▶ `xxx` : `geometry_msgs` 에 소속된 여러 msg 중 하나
- ▶ docs.ros.org ... `geometry_msgs` / `Point32` 검색 시 상세 정보 확인

■ 2. 인스턴스화

- ▶ `msg = Twist()`

```
# 예시
from geometry_msgs.msg \
    import Twist

msg = Twist()
```

Tip – 토픽 타입 바꾸는 방법 (geometry_msgs)

■ 3. 속성 접근

▶ `msg.linear.x = 0.5`

■ 4. CMakeLists.txt 및 package.xml 종속성

```
# CMakeLists.txt
find_package(catkin REQUIRED COMPONENTS
  roscpp
  geometry_msgs
)

# package.xml
<build_depend>geometry_msgs</build_depend>
```

Tip – 구현 전략

- 일단 기능 구현을 하고 개선을 하는 것이 더 빠르다
- 모라이의 `lattice_driving.launch` 를 개선하는 것이 가장 빠른 구현 방법
- 모라이에서 만든 `path` 는 현실에서도 사용 가능하다
- 인지·판단·제어를 나누고 모라이로 알고리즘을 미리 만드는 것도 좋은 방법

- [illegible]

**THANK YOU
FOR YOUR ATTENTION**